Plaatsen en verdiepen in twintig minuten

BIJ labo 5 / hoofdstuk 5

DUUR 20 min klassikaal, daarna aan de opdrachten

Jullie krijgen dit labo een dag vóór de theorie. Dit document geeft je net genoeg om aan de vijf nabouwen te beginnen; de volledige uitleg staat in hoofdstuk 5 van de syllabus (de sectienummers verwijzen daarnaar); lees het deze week alsnog. Bij elk screenshot stel je deze week drie vragen: **is dit een zaak van de container, van één item, of van plaatsing?** De container verdeelt (grid, flexbox), één item mag afwijken (`flex`, een auto-marge), en wat óp iets anders ligt of moet blijven plakken is plaatsing (`position`).

1. Het raster uittekenen: named areas (5.1)

```
body {
  display: grid;
  grid-template-columns: 16rem 1fr;
  grid-template-rows: auto 1fr auto;
  min-height: 100vh;
  grid-template-areas:
    "kop     kop"
    "zijbalk inhoud"
    "voet    voet";
}

header { grid-area: kop; }
nav    { grid-area: zijbalk; }
main   { grid-area: inhoud; }
footer { grid-area: voet; }
```

Lijnnummers (4.6) zijn prima voor één kaart die twee kolommen spant. Voor een grondplan geef je elk item een naam met **`grid-area`** en tekent op de container met **`grid-template-areas`** het raster uit, rij per rij, als een plattegrond. Lees de template hardop en je hoort de pagina. Een naam herhalen laat een gebied uitspreiden; een punt (`.`) is een bewust lege cel. Eén regel: **elk gebied is een rechthoek**. Een L-vorm, of rijen met een ongelijk aantal cellen, maakt de hele declaratie ongeldig en de browser zegt daar niets over (F5.6). **Vuistregel:** nummers voor incidentele plaatsing, areas voor grondplannen.

2. Een rij die mag breken: flex-wrap (5.2)

```
.galerij {
  display: flex;
  flex-wrap: wrap;
  gap: 1.5rem;
}

.galerij > * {
  flex: 1 1 18rem;   /* grow shrink basis */
}
```

Standaard perst flexbox alles op één regel. **`flex-wrap: wrap`** geeft de container toestemming om een nieuwe regel te beginnen. De `flex: 1` van vorige week blijkt een samentrekking van drie waarden: **grow** (hoe gretig het item restruimte opeist), **shrink** (hoe gewillig het inlevert) en **basis** (de startmaat op de hoofdas, de flexbox-eigen vervanger van `width`, F5.7). `flex: 1 1 18rem` zegt: elk item wil zo'n 18rem; past er nog één bij, dan groeien ze samen dicht; past het niet meer, dan breekt de regel. Een galerij die zich aanpast aan elke vensterbreedte, zonder media query. Maar dit blijft **één as**: de tweede regel verdeelt haar eigen ruimte en lijnt niet uit met de eerste (4.2).

3. Eén item wijkt af: de auto-marge (5.3)

```
.balk {
  display: flex;
  align-items: center;
  gap: 1.5rem;
}

.balk .naam {
  margin-right: auto;   /* pushes everything after it to the far end */
}
```

De container beslist voor allemaal, maar één item mag voor zichzelf spreken. **`align-self`** overschrijft `align-items` voor dat ene item. De **auto-marge** is het sluwste instrument: `auto` betekent "verdeel de restruimte hier" (3.6), en in een flex-container slokt ze alle restruimte aan die ene kant op. Naam links, alle links rechts, één declaratie. Hetzelfde idee in een kolom: in een kaart die een flex-column is, krijgt de prijs `margin-top: auto` en staat ze onderaan, hoe lang de tekst erboven ook is.

4. Uit de flow stappen: het ankerduo (5.4, 5.5)

```
.kaart {
  position: relative;   /* anchor: moves nothing, only grants anchor status */
}

.kaart .badge {
  position: absolute;   /* leaves the flow, positions against the card */
  top: 0.75rem;
  right: 0.75rem;
  z-index: 1;           /* above the card image */
}
```

In flow, flexbox en grid overlapt niemand. Voor een badge óp een kaart of vlakjes óp een foto heb je **`position`** nodig. **`relative`** verschuift een element maar laat zijn plek bezet; **`absolute`** haalt het volledig uit de flow (de buren schuiven bij) en rekent vanaf zijn **anker**: de dichtstbijzijnde voorouder met een `position` anders dan `static`. Is er geen, dan rekent het vanaf het document, en dat wil je vrijwel nooit (F5.1). Daarom het vaste duo: `relative` op de ouder, `absolute` op het kind, met `top`, `right`, `bottom` of `left`. Wie bovenop ligt beslist standaard de DOM-volgorde (later ligt hoger); met **`z-index`** neem jij het over, maar alleen op gepositioneerde elementen (F5.3) en met kleine getallen: 1, 2, 10, nooit 9999 (F5.4). En: **positioning is plaatsingsgereedschap, geen layoutgereedschap**. Structuur bouw je met grid en flexbox (F5.5).

5. Blijven plakken: sticky (5.4)

```
header {
  position: sticky;
  top: 0;   /* required: sticky needs a threshold to stick to */
}

html {
  scroll-padding-top: 3.5rem;   /* internal links land below the bar */
}
```

Voor de balk die blijft staan terwijl je scrolt, bestaan twee waarden. **`fixed`** hangt aan het venster en verlaat de flow: de pagina schuift eronder en jij vult het gat met een gegokte `padding-top`. **`sticky`** gedraagt zich als `relative` tot het scrollen het element tegen de opgegeven rand duwt, en plakt daar zolang zijn ouder in beeld is. Geen gat, geen gok. Twee voorwaarden: een drempel (`top: 0`, zonder plakt er niets, F5.2) en een ouder die hoog genoeg is om in te plakken. Interne links laten een kop ónder de balk landen; **`scroll-padding-top`** op `html`, ter grootte van de balk, lost dat op (F5.10).

6. Foto's in het gelid: het drieluik (5.6)

```
.kaart img {
  width: 100%;
  aspect-ratio: 4 / 3;
  object-fit: cover;   /* fill the box, crop what does not fit */
}
```

Het vak heeft een maat, de foto heeft een verhouding, en die botsen: vervorming (F3.8) of een kaart die uit het gelid springt (F4.5). **`aspect-ratio`** geeft de box een vaste verhouding, de hoogte volgt uit de breedte. **`object-fit`** zegt wat de afbeelding met haar vak doet: `fill` (standaard) rekt op, `contain` laat randen open, **`cover`** vult en snijdt bij zonder te vervormen. Drie declaraties, en elk rooster krijgt uniforme foto's, wat de fotograaf ook aanleverde; `aspect-ratio: 1` maakt een vierkant. Dit **drieluik** gaat in je snippets-verzameling.

7. De ouder selecteren: :has() (5.7)

```
.kaart:has(img) {
  padding: 0;   /* image cards: the picture runs to the edges */
}

.kaart:not(:has(img)) {
  border-top: 0.5rem solid var(--color-accent);
}
```

Elke andere selector kijkt omlaag of opzij; **`:has()`** kijkt naar binnen en selecteert een element op basis van wat het bevat. Lees: elke kaart die een afbeelding bevat; elke kaart die er geen bevat (`:not()` ontkent). Geen extra class, geen HTML-wijziging: de stylesheet reageert op de inhoud.

8. Je waarheidsinstrument: de ankervraag (5.8)

Een gepositioneerd element krijgt in het boxmodel-diagram van DevTools een extra ring, *position*. Staat een badge niet waar je ze legde, stel dan de **ankervraag**: klim in de Elements-boom omhoog tot de eerste voorouder met een `position` anders dan `static`. Dat is het element waartegen je badge rekent, en in negen van de tien gevallen is het niet wie je dacht. Voor sticky zijn er **drie vinkjes**: staat er een drempel (`top`)? Heeft de ouder hoogte genoeg? En staat er nergens tussen element en venster een `overflow` die het scrollen kaapt?

Aan de slag

Oefening 1 is punt 1 (een raster van vier bij drie met vier punten voor de lege cellen), oefening 2 is punt 2 (wrap en de drievoudige `flex`; zet daarna je venster op 800 px en kijk), oefening 3 is punt 4 en 6 (het ankerduo met vier vlakjes, twee met een `z-index`, en een vierkante foto via het drieluik), oefening 4 is punt 3 en 5 (de auto-marge op de naam, sticky met `top: 0` en `scroll-padding-top`), oefening 5 is alles samen: het grondplan van punt 1, de balk van punt 3, het ankerduo van punt 4 op elke kaart, het drieluik van punt 6 en `:has()` van punt 7. Stel bij elke nabouw eerst de drie vragen: container, item of plaatsing? Schrijf dan de container, dan de items, en pas op het einde de plaatsing. En zit een badge verkeerd, stel dan eerst de ankervraag vóór je gaat zoeken.